ZERO-PREREQUISITE INSTALLATION & RUNTIME GUIDE

# Slotbook Fresh PC Setup Guide

One-Command Multi-Container Deployment on Brand-New Windows 10/11

| | |
|---|---|
| **TARGET AUDIENCE:** | **REPOSITORY:** |
| Technical Reviewer / Architect | https://github.com/tiruk007/slotbooking.git |
| **HOST PREREQUISITES:** | **TEST VERIFICATION:** |
| Docker Desktop + Git (Zero Node/SQL needed) | **71 of 71 Tests Passing (100%)** |

## 1. Introduction & Overview

Slotbook is an enterprise-grade appointment scheduling engine built with Next.js 14, TypeScript, and PostgreSQL 16. It guarantees zero double-bookings through kernel-level GiST exclusion constraints (`btree_gist`), handles complex timezone mathematics across Daylight Saving Time (DST) boundaries, and eliminates dual-write failures via the Transactional Outbox pattern.

This document provides a complete, foolproof walkthrough for running Slotbook on a **brand-new Windows 10 or 11 laptop or desktop**. No Node.js or PostgreSQL installations are required on the host system; the entire application and database stack are fully containerized.

—

## 2. Windows Base Software Installation

For a fresh Windows machine, install the following standard development tools:

### 2.1 Install Google Chrome (or Edge / Firefox)

Download and install Google Chrome from https://www.google.com/chrome/.

### 2.2 Install Visual Studio Code

Download and install VS Code from https://code.visualstudio.com/. Recommended extensions:

- **Docker** (`ms-azuretools.vscode-docker`)
- **WSL** (`ms-vscode-remote.remote-wsl`)

### 2.3 Install Git for Windows

Download and install Git for Windows from https://git-scm.com/download/win. During installation, when prompted under **Adjusting your line ending conversions**, select: `Checkout as-is, commit Unix-style line endings` (recommended to prevent CRLF script corruption).

### 2.4 Install Docker Desktop for Windows

Download Docker Desktop from https://www.docker.com/products/docker-desktop/. During installation, ensure the checkbox **“Use WSL 2 instead of Hyper-V”** is **CHECKED**. Restart your computer when prompted, then launch Docker Desktop and wait until the whale status icon stays solid green.

—

# 3. Windows Terminal & WSL 2 Verification

Docker Desktop on Windows utilizes the WSL 2 (Windows Subsystem for Linux) backend for near-native Linux kernel performance:

1. Open **PowerShell** as Administrator (Right-click Start -> **Terminal (Admin)** or **PowerShell (Admin)**).
2. Check WSL status:

```
wsl --status
```

3. If WSL is not installed or outdated, run:

```
wsl --install
wsl --update
```

4. In Docker Desktop Settings (`gear icon` -> `General`), ensure **"Use the WSL 2 based engine"** is active.

—

# 4. Repository Cloning & Workspace Setup

Open **Git Bash** or **Windows Terminal** (PowerShell):

```
# 1. Navigate to your desired working directory
cd /c/Users/$USER/Downloads

# 2. Clone the repository
git clone https://github.com/tiruk007/slotbooking.git

# 3. Enter the repository
cd slotbooking
```

—

# 5. PostgreSQL Database Setup (Two Supported Paths)

Slotbook requires PostgreSQL 15+ with the `btree_gist` extension enabled for kernel-level concurrency exclusion.

## Path A (Recommended): Automated Containerized PostgreSQL (Zero Host Install)

If you are using Docker Desktop, **you do NOT need to install or configure PostgreSQL manually on Windows**:

- Docker Compose automatically pulls the official `postgres:16-alpine` image.
- It automatically creates the default database (`slotbook`), sets credentials (`postgres:postgres`), and mounts a named volume (`pgdata`).
- The startup entrypoint script automatically preloads the `btree_gist` extension, executes all migrations (`npm run db:migrate`), and runs seeds (`npm run db:seed`).
- **Host action required**: None! Proceed directly to Section 6.

## Path B (Alternative): Native Local PostgreSQL Installation (Running without Docker)

If running natively on your host machine via Node.js (`npm run dev`):

1. Install PostgreSQL 16 on Windows via EnterpriseDB installer, setting superuser password to `postgres` on port `5432`.
2. In `psql` or pgAdmin, execute:

```
CREATE DATABASE slotbook;
CREATE DATABASE slotbook_test;
\c slotbook;
CREATE EXTENSION IF NOT EXISTS btree_gist;
\c slotbook_test;
CREATE EXTENSION IF NOT EXISTS btree_gist;
```

3. Run migrations and seed data:

```
npm install
npm run db:migrate
npm run db:seed
```

—

# 6. How to Obtain & Configure Environment Variables

The repository includes a ready-to-run `.env.example` template with production-grade local defaults.

Create your local `.env` file:

- In Git Bash or WSL:

```
cp .env.example .env
```

- In PowerShell:

```
Copy-Item .env.example .env
```

## Key Environment Variables Explained

| Variable | Default Value | Where to Obtain / Description |
|---|---|---|
| `DATABASE_URL` | `postgresql://postgres:postgres@localhost:5432/slotbook` | Main PostgreSQL connection string. In Docker, overridden internally to `postgres` container. |
| `TEST_DATABASE_URL` | `postgresql://postgres:postgres@localhost:5432/slotbook_test` | Test database connection string used when running `npm test`. |
| `APP_BASE_URL` | `http://localhost:3000` | Application base URL for self-service guest management links. |
| `NEXT_PUBLIC_APP_URL` | `http://localhost:3000` | Public client URL matching `APP_BASE_URL`. |
| `SESSION_SECRET` | `slotbook_production_secure_secret_key` | Cryptographic secret key for session cookies. See Step 6.1 below. |
| `GMAIL_USER` | `your_email@gmail.com` | Outgoing SMTP Gmail account. See Step 6.2 below. |
| `GMAIL_APP_PASSWORD` | `your_16_character_app_password` | Google App Password (16 chars). See Step 6.2 below. |
| `MAIL_FROM` | `your_email@gmail.com` | Email sender address, matches `GMAIL_USER`. |

### Step 6.1 — How to Generate `SESSION_SECRET`

Generate a cryptographically secure 32-byte secret in any terminal:

```
# In Git Bash / WSL / Linux:
openssl rand -base64 32

# In Windows PowerShell / Node:
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

### Step 6.2 — How to Obtain Google Gmail SMTP Credentials

Slotbook uses Google's secure SMTP relay. Google does not accept standard account passwords; you must generate an App Password:

1. Log into your Google Account at https://myaccount.google.com.
2. Under **Security**, ensure **2-Step Verification** is active.
3. Visit the App Passwords portal: https://myaccount.google.com/apppasswords.

4. Name the application `Slotbook` and click **Create**.
5. Copy the 16-character code (e.g. `abcd efgh ijkl mnop`).
6. Set `GMAIL_USER=your_email@gmail.com` and `GMAIL_APP_PASSWORD=abcdefghijklmnop` in `.env`.

> **Local Mock Mode (No Email Setup Required for Reviewers):**
> If you do not wish to configure a real Gmail account, leave `GMAIL_USER` and `GMAIL_APP_PASSWORD` with default placeholders. The Transactional Outbox stages notifications into PostgreSQL and logs delivery simulations to the terminal. OTP codes are printed directly to the console (`[OTP Service] Code generated: 123456`), allowing 100% of booking and login flows to be verified without real emails.

—

# 7. Single-Command Launch via Docker Compose

In the root of the repository, execute:

```
docker compose up --build
```

### Automated Self-Bootstrapping Sequence

1. **PostgreSQL 16 Engine (`slotbook-postgres`)**:
   - Initializes database cluster and loads the `btree_gist` extension.
   - Mounts a named persistent volume (`pgdata`) so data survives restarts.
   - Activates health check: `pg_isready -U postgres -d slotbook`.
2. **Next.js 14 Application (`slotbook-app`)**:
   - Multi-stage Dockerfile compiles TypeScript and bundles production assets.
   - Executes `docker-entrypoint.sh`:
     - Polls PostgreSQL until healthy.
     - Runs database migrations (`npm run db:migrate`).
     - Runs idempotent seed script (`npm run db:seed`) to create initial host data.
     - Boots the production HTTP server on port 3000.

When the terminal displays:

```
Ready in ...ms
- Local: http://localhost:3000
```

the application is fully operational!

—

# 8. Step-by-Step Verification & Testing Walkthrough

Follow these steps in Google Chrome or any browser to test all core capabilities:

### Test 8.1 — System Health Check

- Open: `http://localhost:3000/api/health`
- Confirms status `200 OK` with database connection verified.

### Test 8.2 — Public Customer Booking Experience

1. Navigate to the seeded host booking page: `http://localhost:3000/book/tirumalesh/30-minutes`
2. **Select a Date**: Click any available calendar date.
3. **Timezone Switcher**: Switch between `Asia/Kolkata` (Host's zone) and `America/New_York` (Guest zone). Slot times project dynamically.
4. **Hold Reservation**: Click an available slot (e.g., 9:10 AM). A 5-minute lease is acquired via PostgreSQL GiST exclusion.
5. **Enter Guest Details**: Enter name and email (e.g. `guest@example.com`).
6. **Verify OTP**: Enter the 6-digit OTP (displayed in console or test sandbox `123456`).

7. **Confirmation**: Redirects to confirmation screen with appointment details and reschedule/cancel links.

### Test 8.3 — Host Private Portal & Availability Management

1. Navigate to Host Login: `http://localhost:3000/host/login`
2. Enter the seeded host credentials:
   - **Email**: `host@slotbook.internal` (or `tirumaleshkommavarapu@gmail.com`)
   - **Password**: `Password123!`
3. **Host Overview**: Inspect confirmed appointments, active event types, and scheduled hours.
4. **Availability Tab**: View weekly recurring rules, add a date override holiday, and click **"Save Weekly Rules"**.
5. **Bookings Tab**: Inspect upcoming, past, and cancelled appointments. Cancel a booking and observe immediate state sync.

### Test 8.4 — Run the Automated Test Suite (71 Tests)

Verify that all 71 unit, integration, and concurrency tests pass against the live PostgreSQL container:

```
docker compose exec slotbook-app npm test
```

Expected result:

```
Test Suites: 7 passed, 7 total
Tests:       71 passed, 71 total
Snapshots:   0 total
Ran all test suites.
```

—

## 9. Common Pitfalls & Windows Troubleshooting

### 9.1 Port 3000 or 5432 Already in Use

- **Symptom**: `listen EADDRINUSE: address already in use :::3000` or `bind: address already in use 0.0.0.0:5432`.
- **Solution**: Kill the conflicting process in PowerShell:

  ```
  Get-Process -Id (Get-NetTCPConnection -LocalPort 3000).OwningProcess | Stop-Process
  Get-Process -Id (Get-NetTCPConnection -LocalPort 5432).OwningProcess | Stop-Process
  ```

  Or change external ports in `docker-compose.yml` (e.g., `"3001:3000"`).

### 9.2 Docker Desktop WSL 2 Engine Not Running

- **Symptom**: `error during connect: This error may indicate that the docker daemon is not running.`
- **Solution**: Launch Docker Desktop from Start menu and wait for the whale icon to turn green. If needed, run `wsl --update` in PowerShell (Admin).

### 9.3 Windows CRLF Line Endings in Shell Scripts

- **Symptom**: `exec /app/docker-entrypoint.sh: no such file or directory` or `\r: command not found`.
- **Solution**: Run `git config core.autocrlf false` followed by `git checkout docker-entrypoint.sh` in Git Bash.

—

## 10. Daily Development Commands & Teardown

| Action | Command |
|---|---|
| Stop Containers (Preserve Data) | `docker compose stop` |
| Restart Stopped Containers | `docker compose start` |
| Tail Live Application Logs | `docker compose logs -f slotbook-app` |
| Tail Database Logs | `docker compose logs -f slotbook-postgres` |

| | |
|---|---|
| Run Migrations Manually | `docker compose exec slotbook-app npm run db:migrate` |
| Re-Seed Database | `docker compose exec slotbook-app npm run db:seed` |
| Full Teardown (Clean Slate) | `docker compose down -v` |

—

**Slotbook Production Scheduling Engine**

Authored by Tirumalesh Kommavarapu | Verified Repository: github.com/tiruk007/slotbooking